Supporting Multiple .NET Versions in a Class Library Project

Jason Roberts

.NET Developer

@robertsjason | dontcodetired.com

Module Overview

An overview of multi-targeting

Conditional compilation preprocessor directives

Add multi-targeting to a class library project

Conditional property groups

The effect of multi-targeting on project properties

Add conditional code for different targets

Create console apps for different targets

Preventing bugs if new targets are added

Dealing with unsupported operations for some targets

An Overview of Multi-targeting

- Output multiple DLL files from a single project
- Each DLL supports (“targets”) a different .NET version
- Makes class library more widely consumable
- Compile different C# code for different targets (.dll)
- Support different targets for different apps inside your organization
- Support different targets for open source libraries on NuGet.org
- Access version/target-specific APIs

Target Framework Moniker (TFM)

A standardized “code” that represents a target framework.

TFM Examples

Framework & version	TFM
.NET Framework 4.8	net48
.NET Core 3.1	netcoreapp3.1
.NET Standard 2.1	netstandard2.1
.NET 5	net5.0
.NET 6	net6.0

OS-specific TFMs

Allow APIs from a specified operating system to be available

Introduced from .NET 5 and onwards

Examples:

- net6.0-windows
- net6.0-ios
- net6.0-ios15.1

Conditional Compilation Preprocessor Directives

Allow you to include or exclude source code from the compilation process based on whether or not specific preprocessor symbols have been defined.

Preprocessor Symbol: a named text “flag” that can either exist (“defined”) or not exist (“undefined”), i.e. a Boolean value.

Conditional Compilation Preprocessor Directives

#if	#elif	#else	#endif
Starts a conditional compilation block	“else if” Alternate conditional block	Alternate block if other blocks not matched	Ends a conditional compilation block

Conditional Compilation Preprocessor Directives

```
#if DEBUG
        Console.WriteLine("DEBUG is defined");
#else
        Console.WriteLine("DEBUG is not defined");
#endif

// The DEBUG preprocessor symbol is automatically defined during debug builds
```

#error

Generates a compiler error with a custom message.

Auto-defined Symbols for TFMs

net48	netstandard2.1	net6.0
NET48 **NETFRAMEWORK**	**NETSTANDARD2_1** **NETSTANDARD**	**NET6_0** **NET**

Additional Symbols (.NET 5+)

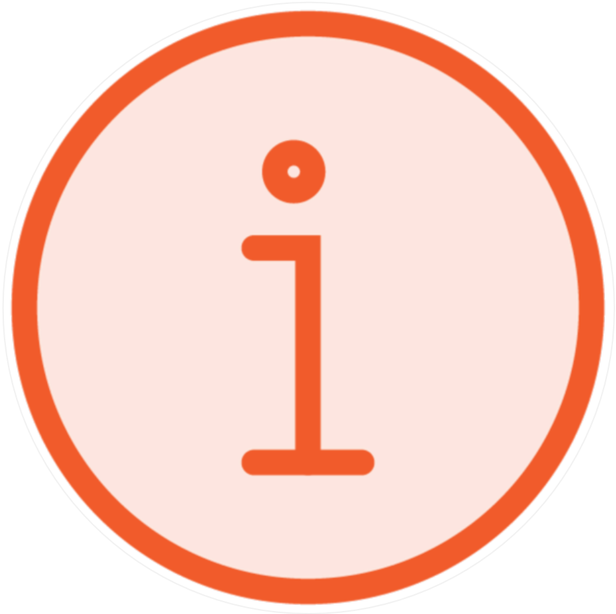
NET48_OR_GREATER, NET472_OR_GREATER, NET471_OR_GREATER, NET47_OR_GREATER, NET462_OR_GREATER, NET461_OR_GREATER, NET46_OR_GREATER, NET452_OR_GREATER, NET451_OR_GREATER, NET45_OR_GREATER, NET40_OR_GREATER, NET35_OR_GREATER, NET20_OR_GREATER

Additional Symbols (.NET 5+)

NETSTANDARD2_1_OR_GREATER,
NETSTANDARD2_0_OR_GREATER,
NETSTANDARD1_6_OR_GREATER,
NETSTANDARD1_5_OR_GREATER,
NETSTANDARD1_4_OR_GREATER,
NETSTANDARD1_3_OR_GREATER,
NETSTANDARD1_2_OR_GREATER,
NETSTANDARD1_1_OR_GREATER,
NETSTANDARD1_0_OR_GREATER

Additional Symbols (.NET 5+)

NET6_0_OR_GREATER,
NET5_0_OR_GREATER,
NETCOREAPP3_1_OR_GREATER,
NETCOREAPP3_0_OR_GREATER,
NETCOREAPP2_2_OR_GREATER,
NETCOREAPP2_1_OR_GREATER,
NETCOREAPP2_0_OR_GREATER,
NETCOREAPP1_1_OR_GREATER,
NETCOREAPP1_0_OR_GREATER



TFM / Symbol Documentation

https://docs.microsoft.com/en-us/dotnet/standard/frameworks

https://bit.ly/tfmdocs

Module Summary

An overview of multi-targeting and TFMs

#if #elif #else #endif

<TargetFrameworks>...</TargetFrameworks>

<PropertyGroup Condition="...">

Multi-targeting in project properties

#if NET6_0

Created console apps for different targets

Dealing with unsupported operations

- #error
- PlatformNotSupportedException
- IsWriteAsHexSupported

Up Next:

Unit Testing Class Library Projects